

Лабораторна робота № 3

Тема: «Дослідження безпечної реалізації та експлуатації децентралізованих додатків».

Мета роботи: отримання навичок роботи із децентралізованими додатками та оцінка безпеки інформації при їх функціонуванні.

Виконав: ФБ-52мп Карабінський Василь

Завдання на лабораторну роботу. Для першого типу лабораторних робіт: дослідження вимог OWASP (безпека веб-додатків) та складання аналогічних вимог для обраної системи децентралізованих додатків. Обрана система: Ethereum та EVM-сумісні платформи (Polygon, Arbitrum, Optimism, BSC, Avalanche C-Chain).

1. Вибір системи та контекст дослідження

Для дослідження обрано Ethereum та сумісні з ним EVM-платформи. Цей вибір не випадковий: близько 60% усього TVL (Total Value Locked) у децентралізованих фінансах сконцентровано саме в Ethereum-екосистемі. Це робить її як найбільшою мішенню для атак, так і найбільш дослідженою з точки зору безпеки. Усі EVM-сумісні мережі (Polygon, Arbitrum, Optimism, BSC, Avalanche) виконують ту саму віртуальну машину, тому виявлені вразливості та запропоновані захисти переносяться між ними практично без змін.

За оцінками Rekt News, з 2016 по 2024 рік сукупні втрати від атак на dApp у Ethereum-екосистемі склали понад 5 мільярдів доларів. Це робить безпеку смарт-контрактів не просто академічним питанням, а ключовим виробничим обмеженням, що визначає життєздатність будь-якого DeFi-протоколу.

2. Огляд галузевих стандартів безпеки

За аналогією з OWASP у світі веб-додатків, для смарт-контрактів сформувалося кілька паралельних стандартів. Кожен з них покриває певний рівень абстракції, тож на практиці використовуються разом.

2.1. OWASP Top 10

Open Web Application Security Project це некомерційна організація, що з 2001 року веде список найкритичніших вразливостей веб-додатків. Поточна версія OWASP Top 10:2021 містить десять категорій від A01 (Broken Access Control) до A10 (SSRF) і використовується як галузевий мінімум. Окремо

існує детальніший стандарт ASVS з понад 280 формальними вимогами для верифікації при аудиті.

2.2. SWC Registry

Smart Contract Weakness Classification це прямий аналог CWE для смарт-контрактів. Містить понад 130 класифікованих вразливостей з ідентифікаторами SWC-100 і вище, кожна з описом, прикладами уразливого коду та посиланнями на CVE. Статичні аналізатори (Slither, Mythril, Securify) використовують SWC як базу для маркування знайдених проблем.

2.3. DASP Top 10 та SCSVS

Decentralized Application Security Project Top 10 від NCC Group (2018) прямо моделюється на OWASP Top 10 і містить десять класів вразливостей смарт-контрактів. Smart Contract Security Verification Standard (SCSVS) це прямий аналог OWASP ASVS з понад 130 конкретними вимогами для формального аудиту.

2.4. ConsenSys Best Practices та Trail of Bits

Окрім формальних стандартів, у галузі сформувалися два авторитетних посібники: ConsenSys Smart Contract Best Practices (відкритий ресурс з рекомендаціями) та Trail of Bits Building Secure Contracts (документація з провідної аудиторської фірми, опублікована як open-source). Обидва є де-факто стандартами індустрії.

3. Прямий mapping OWASP Top 10 у світ смарт-контрактів

Не кожна категорія OWASP має прямий аналог у смарт-контрактах. Сім із десяти переносяться напряду (особливо Access Control, Crypto Failures, Auth Failures, Logging Failures). Категорії A03 (Injection) і A10 (SSRF) не мають прямого еквівалента, бо породжені серверною архітектурою, якої у блокчейні не існує. Зведений mapping наведено в таблиці нижче.

OWASP A0X	Аналог у смарт-контракті	SWC ID	Базова вимога
A01: Broken Access Control	Незахищена ownership-передача, tx.origin замість msg.sender, default function visibility	SWC-100, 105, 115	OpenZeppelin AccessControl/Ownable, тільки msg.sender, явний visibility
A02: Cryptographic Failures	Генерація випадковості через block.timestamp, ECDSA malleability, replay підписів	SWC-117, 120, 121	Chainlink VRF, EIP-712 з nonce та chainId, OpenZeppelin ECDSA.sol
A03: Injection	Прямого аналога немає, найближче delegatecall до недовіреної адреси	SWC-112	Заборонити arbitrary external calls, allowlist цільових адрес
A04: Insecure Design	Порушення Checks-Effects-Interactions, що веде до	SWC-107	CEI-патерн, OpenZeppelin ReentrancyGuard

OWASP A0X	Аналог у смарт-контракті	SWC ID	Базова вимога
	reentrancy		
A05: Security Misconfiguration	Floating pragma, старий компілятор, помилкове ім'я конструктора	SWC-102, 103, 118	pragma solidity 0.8.20 (fixed), компілятор не нижче 0.8.x
A06: Vulnerable Components	Застарілі версії OpenZeppelin, форки невідомого походження	не класифіковано	Імпорт з прм з фіксованою версією, підписка на security advisories
A07: Authentication Failures	tx.origin authentication, відсутність replay protection	SWC-115, 121	msg.sender + nonce, EIP-712 для meta-tx
A08: Software/Data Integrity	Не захищена upgradeability, SELFDESTRUCT без захисту, delegatecall hijacking	SWC-106, 112	Timelock + Multisig для upgrade, заборона SELFDESTRUCT
A09: Logging/Monitoring	Відсутність events на критичних змінах стану	не класифіковано	events на всі state changes, моніторинг через Forta/Defender
A10: Server-Side Request Forgery	Прямого аналога немає, найближче arbitrary external calls	SWC-112	Allowlist для external calls, заборона користувацьких target-адрес

4. Авторський перелік 10 категорій вимог безпеки для Ethereum dApp

На основі OWASP Top 10, SWC Registry, DASP та сучасних кейсів атак сформовано власний перелік з 10 категорій, специфічних для Ethereum dApp. Перш ніж переходити до детального опису кожної, варто продивитись історію втрат від реальних атак. Графік нижче ілюструє масштаб проблеми.

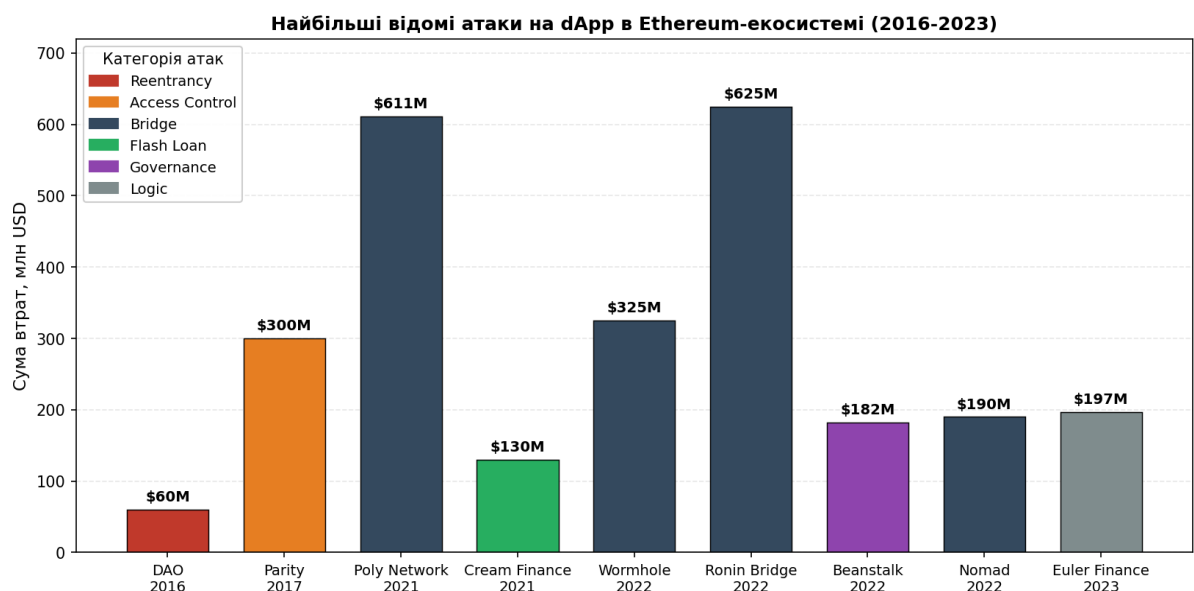


Рис. 1. Найбільші відомі атаки на dApp у Ethereum-екосистемі, 2016-2023

Звертає на себе увагу те, що чотири з топ-5 інцидентів за сумою втрат припадають на крос-чейн мости, а не на самі контракти Ethereum. Це підкреслює важливість бридж-безпеки як окремої категорії, чого зазвичай

немає в класичних OWASP-mapping. Категорії далі впорядковано не за алфавітом і не за SWC ID, а за частотою появи у реальних інцидентах 2020-2024 років.

4.1. Категорія 1: Reentrancy та порушення CEI-патерну

Reentrancy це класична вразливість смарт-контрактів, відома з 2016 року після атаки на The DAO. Незважаючи на десятиліття усвідомлення проблеми, вона досі регулярно з'являється у нових протоколах, особливо там, де відбуваються зовнішні виклики до недовірених контрактів. У 2023 році з'явився підклас read-only reentrancy, що зачепив Curve, Balancer та ще кілька великих протоколів, тож проблема залишається актуальною.

Імовірність експлуатації	Висока, особливо в DeFi-кредитуванні та сховищах
Технічний імпакт	Критичний (повне спустошення коштів)
Бізнес-імпакт	Катастрофічний
Стандарти класифікації	SWC-107, CWE-841
Інструменти виявлення	Slither (виявляє автоматично), Mythril, Echidna для invariant-тестування

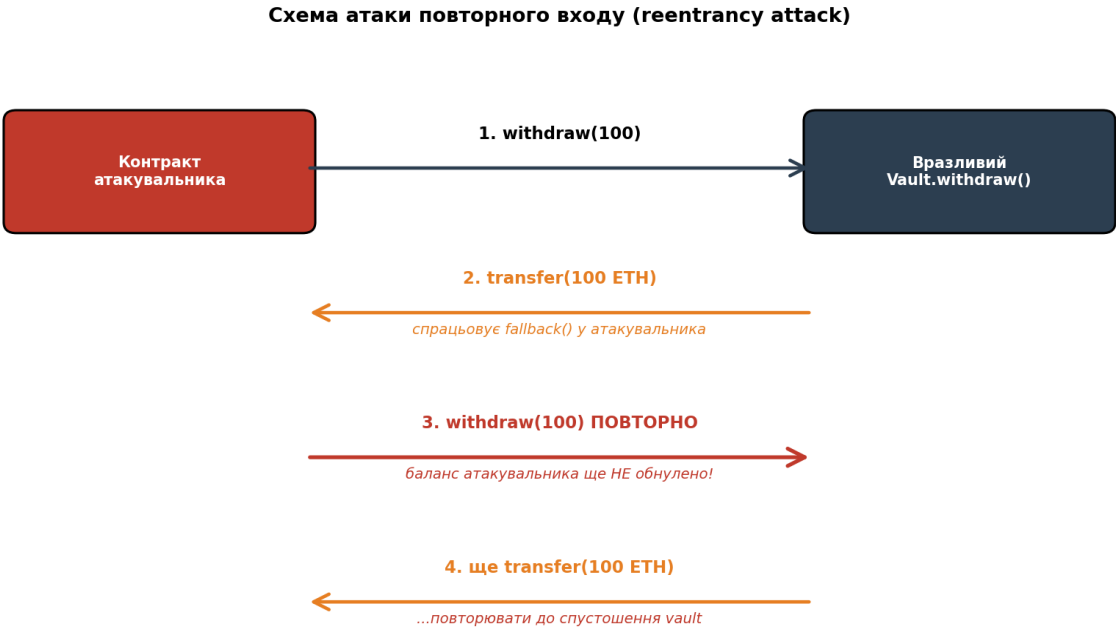


Рис. 2. Принцип атаки повторного входу

Сценарії експлуатації

Сценарій 1 (класичний). Контракт-vault зменшує баланс користувача після виклику transfer(). Атакувальник у своєму fallback() рекурсивно викликає withdraw(), і кожен наступний виклик бачить старий, ще не оновлений баланс.

Сценарій 2 (cross-function). Атакувальник через fallback викликає не ту саму функцію, а іншу, що ділить стан з першою. Наприклад, `withdraw()` викликає `transfer()`, який бачить стару кількість токенів. `ReentrancyGuard`, навішений лише на `withdraw()`, у такому випадку не допомагає.

Сценарій 3 (read-only). Атакувальник не змінює стан, але через `reentrancy state`-функція тимчасово повертає невірне значення, яке інший протокол використовує для розрахунків (наприклад, оракул, що читає резерви Curve під час свопу). Атаки 2022-2023 років на Curve та Balancer пов'язані саме з цим.

Приклад уразливого коду

```
// УРАЗЛИВИЙ варіант
contract SimpleVault {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint256 amount) public {
        require(balances[msg.sender] >= amount, "Insufficient");
        // ПОМИЛКА: спочатку transfer, потім оновлення стану
        (bool ok, ) = msg.sender.call{value: amount}("");
        require(ok, "Transfer failed");
        balances[msg.sender] -= amount;
    }
}
```

Захищений варіант з CEI-патерном та ReentrancyGuard:

```
// SPDX-License-Identifier: MIT
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract SecureVault is ReentrancyGuard {
    mapping(address => uint256) public balances;

    function withdraw(uint256 amount) public nonReentrant {
        require(balances[msg.sender] >= amount, "Insufficient");
        // CEI: спочатку Check, потім Effect (оновлення стану)
        balances[msg.sender] -= amount;
        // ...і тільки потім Interaction (зовнішній виклик)
        (bool ok, ) = msg.sender.call{value: amount}("");
        require(ok, "Transfer failed");
    }
}
```

Вимоги для категорії

1. Завжди дотримуватись патерну Checks-Effects-Interactions: всі перевірки + усі зміни стану ДО будь-яких зовнішніх викликів.
2. Застосовувати OpenZeppelin `ReentrancyGuard` на функціях з `external calls`, особливо з `.call{value: ...}`.

3. Для протидії cross-function reentrancy використовувати спільний модифікатор на групі пов'язаних функцій.
4. Тестувати на read-only reentrancy через invariant-тести в Foundry або Echidna.

4.2. Категорія 2: Контроль доступу до критичних функцій

За статистикою аудиторських фірм Trail of Bits та ConsenSys Diligence, помилки контролю доступу залишаються найпоширенішою категорією вразливостей у новостворених контрактах. У 2017 році саме незахищена ініціалізація бібліотеки коштувала Parity Multisig понад 300 мільйонів доларів. Особливо болюче, що ці помилки часто тривіальні: забутий модифікатор, неправильний default visibility, плутанина між tx.origin та msg.sender.

Імовірність експлуатації	Дуже висока (#1 за частотою у нових контрактах)
Технічний імпакт	Критичний (повне захоплення контракту)
Бізнес-імпакт	Критичний/Катастрофічний
Стандарти класифікації	SWC-100, SWC-105, SWC-115, CWE-284, CWE-285
Інструменти виявлення	Slither (виявляє відсутність modifier), Aderyn, ручний аудит

Сценарії експлуатації

Сценарій 1 (незахищена ініціалізація). У проху-патерні логічний контракт має функцію initialize() замість конструктора. Якщо вона не позначена як onlyOnce, будь-хто може ініціалізувати контракт повторно з власною адресою як owner. Саме це сталося з Parity (2017) і Wormhole (2022).

Сценарій 2 (tx.origin замість msg.sender). Адмін-функція використовує require(tx.origin == owner). Атакувальник створює фішинговий сайт, що змушує власника підписати начебто безпечну транзакцію до зловмисного контракту, який сам викликає вразливий контракт. Перевірка tx.origin проходить, бо власник початково підписав транзакцію, і захоплення відбувається.

Сценарій 3 (default visibility). До Solidity 0.5 функції без явного visibility були public за замовчуванням. Звичайна помилка з падінням до позиції внутрішньої функції робила її доступною ззовні. Хоча в сучасних версіях компілятор вимагає явного visibility, у legacy-контрактах це досі зустрічається.

Приклад уразливого коду

```
// УРАЗЛИВИЙ варіант: будь-хто може завершити аукціон
contract AuctionHouse {
```

```

address public highestBidder;
uint256 public highestBid;
bool public ended;

// Немає модифікатора, видимість public за замовчуванням
function endAuction() public {
    require(!ended, "Already ended");
    ended = true;
    payable(highestBidder).transfer(highestBid);
}
}

```

Захищений варіант з OpenZeppelin Ownable:

```

import "@openzeppelin/contracts/access/Ownable.sol";

contract SecureAuctionHouse is Ownable {
    address public highestBidder;
    uint256 public highestBid;
    bool public ended;
    uint256 public auctionEndTime;

    constructor() Ownable(msg.sender) {}

    function endAuction() external onlyOwner {
        require(block.timestamp >= auctionEndTime, "Too early");
        require(!ended, "Already ended");
        ended = true;
        (bool ok, ) = payable(highestBidder).call{value: highestBid}("");
        require(ok, "Transfer failed");
        emit AuctionEnded(highestBidder, highestBid);
    }

    event AuctionEnded(address indexed winner, uint256 amount);
}

```

Вимоги для категорії

5. Використовувати OpenZeppelin Ownable для простих систем або AccessControl для ролей; ніколи не писати власні modifier для авторизації з нуля.
6. Для авторизації застосовувати тільки msg.sender, ніколи tx.origin.
7. Явно вказувати visibility (external/public/internal/private) для кожної функції та змінної стану.
8. Для гроху-контрактів захищати функцію initialize() через модифікатор initializer з OpenZeppelin.
9. Для адмін-функцій використовувати мультипідпис (Gnosis Safe мінімум 2-of-3, у виробничих DeFi-протоколах 4-of-7).

4.3. Категорія 3: Маніпуляція ціновими оракулами

DeFi-протоколи постійно потребують зовнішніх даних: ціни активів для розрахунку застави, курси для свопів, ставок для деривативів. Оскільки блокчейн ізольований, ці дані подаються через оракули. Якщо контракт

довіряє оракулу, який легко маніпулювати (наприклад, спот-ціні на одному DEX), він стає вразливим. З 2020 року це призвело до більш ніж десятка атак з сукупними втратами понад 500 мільйонів доларів.

Імовірність експлуатації	Висока (поширений сценарій у lending-протоколах)
Технічний імпакт	Критичний (необмежені позики, ліквідації)
Бізнес-імпакт	Катастрофічний
Стандарти класифікації	не має прямого SWC, пов'язане з SWC-114
Інструменти виявлення	Складно для статичного аналізу, потрібен manual review архітектури

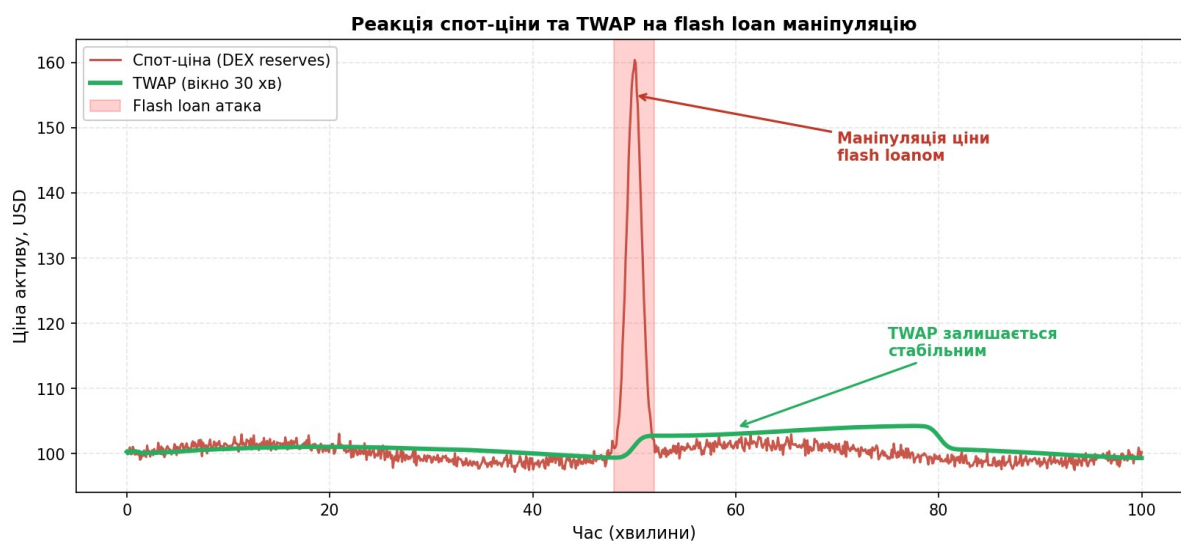


Рис. 3. Стійкість TWAP проти спот-ціни під час flash loan атаки

Сценарії експлуатації

Сценарій 1 (spot price manipulation). Атакувальник бере flash loan на 50 мільйонів USDC, обмінює на токен у відносно невеликому пулі Uniswap, ціна токenu зростає в 3-5 разів. Lending-протокол, що використовує цей пул як oracle, бачить високу ціну і дозволяє атакувальнику взяти позику під переоцінену заставу. Атакувальник повертає flash loan і залишається з різницею. Класичний приклад це bZx (2020).

Сценарій 2 (stale price). Оракул не оновлюється з технічних причин: наприклад, L2-секвенсер у Arbitrum або Optimism впав на кілька годин. Контракти, що читають Chainlink price feed, отримують стару ціну, поки реальний ринок уже впав. Це дозволяє арбітражні атаки проти таких контрактів.

Сценарій 3 (oracle censorship). Operator або кворум валідаторів Chainlink може теоретично відмовитись оновлювати ціни конкретного активу

в критичний момент. Хоча для основних feeds це малоймовірно через децентралізацію, для нішевих активів це реальний ризик.

Приклад уразливого коду

```
// УРАЗЛИВИЙ варіант: спот-ціна з Uniswap V2 reserves
import "./IUniswapV2Pair.sol";

contract VulnerableLendingPool {
    IUniswapV2Pair public pair;

    function getTokenPrice() public view returns (uint256) {
        (uint112 r0, uint112 r1, ) = pair.getReserves();
        // Атакувальник з flash loan легко зсуне ці резерви
        return uint256(r1) * 1e18 / uint256(r0);
    }
}
```

Захищений варіант з Chainlink + перевірка свіжості:

```
import "@chainlink/contracts/src/v0.8/AggregatorV3Interface.sol";

contract SecureLendingPool {
    AggregatorV3Interface public priceFeed;
    uint256 constant MAX_AGE = 3600; // 1 година

    function getTokenPrice() public view returns (uint256) {
        (, int256 price, , uint256 updatedAt, ) = priceFeed.latestRoundData();
        require(price > 0, "Invalid price");
        require(block.timestamp - updatedAt < MAX_AGE, "Stale price");
        return uint256(price);
    }
}
```

Вимоги для категорії

10. Ніколи не використовувати спот-ціну DEX (Uniswap getReserves, Sushiswap balanceOf) як oracle для протоколу з фінансовою цінністю.
11. Як основний oracle використовувати Chainlink або Pyth. Для додаткової стійкості агрегувати кілька джерел через медіану.
12. Завжди перевіряти timestamp оновлення (updatedAt) і відхиляти стару ціну (понад 1 годину для волатильних активів).
13. Для протоколів, які мусять використовувати DEX як oracle, застосовувати TWAP з вікном не менше 30 хвилин (тоді flash loan не зможе зсунути результат).
14. Передбачити circuit breaker: якщо ціна змінилась на 20%+ за блок, призупинити контракт до ручної перевірки.

4.4. Категорія 4: MEV та атаки через публічний мемпул

Транзакції в Ethereum не виконуються одразу, спочатку вони потрапляють у публічний мемпул, де чекають на включення в блок. Боти можуть бачити ці транзакції до їх виконання і вставляти власні з вищою

комісією газу. Це породжує цілу екосистему MEV (Maximal Extractable Value), яка за оцінками Flashbots відбирає у звичайних користувачів понад 700 мільйонів доларів на рік через sandwich-атаки та front-running.

Імовірність експлуатації	Надзвичайно висока (відбувається на кожному великому свопі)
Технічний імпакт	Середній (втрата ефективності, не повний злам)
Бізнес-імпакт	Низький-Середній (індивідуально), високий (системно)
Стандарти класифікації	SWC-114, не має CWE-аналога
Інструменти виявлення	Виявлення через моніторинг мемпулу (mempool.guru, Eden Network)

Сценарії експлуатації

Сценарій 1 (sandwich attack). Користувач відправляє swap на велику суму. Бот бачить транзакцію в мемпулі, купує той самий актив першим (front-run), ціна зростає, виконується транзакція користувача за гіршою ціною, бот продає назад (back-run) і забирає різницю.

Сценарій 2 (NFT mint front-running). При популярному дропі мінт коштує однакову суму газу для всіх, але цінні токени з'являються першими. Бот платить вищу комісію газу, щоб опинитись на початку черги мінту і отримати рідкісні токени.

Сценарій 3 (arbitrage theft). Користувач знайшов прибуткову арбітражну можливість між двома DEX. Бот аналізує його транзакцію в мемпулі, копіює логіку і виконує арбітраж раніше з вищим gas price.

Вимоги для категорії

15. На фронтенді dApp розраховувати amountOutMin за поточною ціною мінус 0,5-1% slippage. Якщо бот зсуне ціну більше, транзакція користувача просто відхилиться.
16. Інтегрувати Flashbots Protect або CoW Protocol для відправки приватних транзакцій поза публічним мемпулом.
17. Для критичних операцій (наприклад, рідкісних мінтів) використовувати commit-reveal схему: спочатку фіксація хешу, потім розкриття.
18. Для DAO-голосувань і ліквідацій використовувати batch auctions замість real-time виконання, щоб усі транзакції в одному блоці отримували однакову ціну.

4.5. Категорія 5: Криптографічні підписи та replay-атаки

Багато протоколів дозволяють користувачам підписувати повідомлення off-chain, а потім хтось інший (relayer) надсилає їх у контракт. Це економить газ і дозволяє gasless-операції (meta-transactions, permits, vouchers). Якщо підпис побудовано неправильно, можлива повторна його експлуатація: атакувальник може використати один валідний підпис двічі або перенести з однієї мережі в іншу.

Імовірність експлуатації	Висока (повсюдно в meta-transactions та ERC-2612 permits)
Технічний імпакт	Критичний (повне виведення коштів)
Бізнес-імпакт	Критичний
Стандарти класифікації	SWC-117, SWC-121, SWC-122, CWE-294
Інструменти виявлення	Slither (виявляє use of ecrecover без EIP-712), manual audit

Сценарії експлуатації

Сценарій 1 (cross-chain replay). Користувач підписує дозвіл на переказ в Ethereum mainnet. Розробник не включив chainId у підписане повідомлення. Атакувальник бере цей самий підпис і відправляє в ідентичний контракт у Polygon, де користувач теж має кошти. Гроші зникають з двох мереж за один підпис.

Сценарій 2 (signature malleability). ECDSA має математичну особливість: для одного дійсного підпису (v, r, s) існує другий, еквівалентний підпис (v', r, s'), де $s' = \text{curve_order} - s$. Якщо контракт зберігає використані підписи в mapping і не нормалізує їх, атакувальник генерує другий варіант і викликає функцію вдруге.

Сценарій 3 (missing nonce). Користувач підписує "чек" на переказ 100 tokenів. Якщо в підписі немає порядкового номера (nonce), атакувальник може відправляти цей самий чек у контракт нескінченно, поки не виведе весь баланс.

Приклад уразливого коду

```
// УРАЗЛИВИЙ варіант: немає nonce, немає chainId, є malleability
contract VoucherClaim {
    mapping(bytes32 => bool) public used;

    function claim(
        address to, uint256 amount,
        uint8 v, bytes32 r, bytes32 s
    ) external {
        bytes32 hash = keccak256(abi.encodePacked(to, amount));
        address signer = ecrecover(hash, v, r, s);
        require(signer == owner, "Invalid");
        require(!used[keccak256(abi.encode(v, r, s))], "Used");
        used[keccak256(abi.encode(v, r, s))] = true;
        token.transfer(to, amount);
    }
}
```

```
}  
}
```

Захищений варіант через EIP-712 + nonce + OpenZeppelin ECDSA:

```
import "@openzeppelin/contracts/utils/cryptography/EIP712.sol";  
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";  
  
contract SecureVoucherClaim is EIP712 {  
    using ECDSA for bytes32;  
    mapping(uint256 => bool) public usedNonces;  
  
    bytes32 private constant VOUCHER_TYPEHASH = keccak256(  
        "Voucher(address to,uint256 amount,uint256 nonce,uint256 deadline)"  
    );  
  
    constructor() EIP712("VoucherSystem", "1") {}  
  
    function claim(address to, uint256 amount, uint256 nonce,  
        uint256 deadline, bytes calldata sig) external {  
        require(block.timestamp <= deadline, "Expired");  
        require(!usedNonces[nonce], "Nonce used");  
        usedNonces[nonce] = true;  
  
        bytes32 structHash = keccak256(abi.encode(  
            VOUCHER_TYPEHASH, to, amount, nonce, deadline));  
        bytes32 digest = _hashTypedDataV4(structHash);  
        require(digest.recover(sig) == owner, "Invalid signature");  
  
        token.transfer(to, amount);  
    }  
}
```

Вимоги для категорії

- 19.Завжди використовувати EIP-712 для structured data signing замість сирого keccak256 + ecrecover. EIP-712 автоматично включає chainId та адресу контракту в підпис.
- 20.Включати nonce у кожне підписане повідомлення. Користуватись OpenZeppelin Nonces для коректної реалізації.
- 21.Включати deadline (термін дії) у підпис, щоб старі підписи не могли бути використані.
- 22.Використовувати OpenZeppelin ECDSA.sol замість сирого ecrecover. Бібліотека автоматично перевіряє malleability.

4.6. Категорія 6: DoS та виснаження газу

Ethereum накладає на блок жорсткий ліміт газу (30 мільйонів одиниць станом на 2024 рік). Якщо функція потребує більше газу, ніж дозволено в одному блоці, вона ніколи не зможе виконатись. Атакувальник може навмисно створити умови, за яких критична функція стане неможливою для виклику, або одна транзакція в циклі завжди завершуватиметься помилкою.

Імовірність експлуатації	Висока (особливо у функціях розподілу винагород)
Технічний імпакт	Високий (заморожування коштів, неможливість виконати функцію)
Бізнес-імпакт	Високий
Стандарти класифікації	SWC-113, SWC-126, SWC-128, CWE-400
Інструменти виявлення	Slither (виявляє unbounded loops), Mythril, manual review

Сценарії експлуатації

Сценарій 1 (block gas limit DoS). Контракт зберігає масив користувачів і має функцію, яка проходить циклом for по всіх і розподіляє винагороду. Атакувальник створює багато порожніх адрес і реєструється кожною. Розмір масиву зростає, цикл починає вимагати понад 30 мільйонів газу і функція ламається назавжди.

Сценарій 2 (push-pattern with revert). Контракт виплачує переможцям аукціону через msg.sender.transfer() у циклі. Атакувальник створює контракт з fallback(), що завжди робить revert. Коли система намагається переказати йому кошти, вся транзакція відкочується, і ніхто не отримує виплат.

Сценарій 3 (insufficient gas griefing). У relayed transactions атакувальник навмисно надсилає недостатньо газу, щоб внутрішній виклик завершився помилкою, але зовнішня транзакція пройшла. Це дозволяє маніпулювати станом контракту нестандартним чином.

Приклад уразливого коду

```
// УРАЗЛИВИЙ варіант: push-pattern з циклом
contract BadAirdrop {
    address[] public users;

    function distribute() external {
        for (uint256 i = 0; i < users.length; i++) {
            // Якщо один transfer провалиться, провалюються усі
            payable(users[i]).transfer(1 ether);
        }
    }
}
```

Захищений варіант з pull-pattern:

```
contract SafeAirdrop {
    mapping(address => uint256) public pending;

    function allocate(address[] calldata users, uint256 amount) external onlyOwner {
        for (uint256 i = 0; i < users.length; i++) {
            pending[users[i]] += amount;
        }
    }

    // Кожен користувач сам забирає свою частку
    function claim() external {
```

```

    uint256 amount = pending[msg.sender];
    require(amount > 0, "Nothing to claim");
    pending[msg.sender] = 0;
    (bool ok, ) = msg.sender.call{value: amount}("");
    require(ok, "Transfer failed");
  }
}

```

Вимоги для категорії

23. Уникати unbounded loops над колекціями, розмір яких залежить від користувачів.
24. Використовувати pull-pattern (claim()) замість push-pattern (distribute()).
25. Для перевірки наявності елемента застосовувати mapping (константний час) замість пошуку по масиву.
26. Перевіряти gasleft() перед external calls, які мають критичне значення для подальшої логіки.

4.7. Категорія 7: Безпека upgradeability та storage collisions

Хоча в ідеальному світі смарт-контракт після розгортання незмінний, на практиці більшість великих протоколів використовують проху-патерни для оновлення коду. Найпоширеніші: Transparent Proxy, UUPS, Beacon Proxy. Кожен патерн має свої компроміси між безпекою та зручністю, але всі вони додають нову поверхню атак. Помилки в storage layout або в правах на upgrade призвели до десятків інцидентів, включно з Wormhole (2022).

Імовірність експлуатації	Середня (помилки при оновленні)
Технічний імпакт	Критичний (заміна коду на зловмисний)
Бізнес-імпакт	Катастрофічний
Стандарти класифікації	не має прямого SWC, пов'язане з SWC-118
Інструменти виявлення	OpenZeppelin Upgrades plugin, slither-check-upgradeability

Сценарії експлуатації

Сценарій 1 (storage collision). Розробник оновлює логічний контракт і додає нову змінну в середину списку, а не в кінець. Оскільки проху зберігає стан у фіксованих слотах за порядком оголошення, нова змінна записується поверх існуючих даних. Користувачі починають бачити чужі баланси, виходить тотальний хаос.

Сценарій 2 (unprotected upgradeTo). Функція upgradeTo() доступна тільки одному ЕОА (звичайному гаманцю розробника). Атакувальник через соціальну інженерію або компрометацію девайсу отримує приватний ключ і замінює код на свій. Усі кошти протоколу виводяться в одну транзакцію.

Сценарій 3 (uninitialized implementation). У UUPS-патерні логічний контракт містить функцію upgradeTo, що не повинна викликатись напрямку, тільки через проху. Якщо implementation не ініціалізовано, атакувальник викликає initialize() на ньому, стає власником і викликає selfdestruct, ламаючи проху. Так сталося з Parity Multisig у 2017 році.

Вимоги для категорії

27. Використовувати OpenZeppelin Upgrades Plugins, який автоматично перевіряє storage layout перед оновленням.
28. Нові змінні стану додавати тільки в кінець, ніколи не змінювати порядок існуючих.
29. Права на upgradeTo надавати тільки мультипідпису (мінімум 4-of-7) з обов'язковим timelock 48-72 години.
30. Для UUPS захищати implementation contract від ініціалізації через _disableInitializers() у конструкторі.
31. Документувати всі поля сховища (storage gaps) для майбутніх версій.

4.8. Категорія 8: Атаки на on-chain governance

Багато DeFi-протоколів керуються через DAO-голосування власників token-ів. Це створює власну поверхню атак. Атака на Beanstalk у квітні 2022 року стала уроком для всієї індустрії: атакувальник за 13 секунд провів зловмисну пропозицію, використавши flash loan для тимчасового набуття більшості голосів. Інша частина категорії це централізаційні бекдори, які розробники залишають "про всяк випадок".

Імовірність експлуатації	Середня (зростає з ускладненням протоколів)
Технічний імпакт	Критичний (повна зміна параметрів протоколу)
Бізнес-імпакт	Катастрофічний
Стандарти класифікації	не має прямого SWC, пов'язане з логікою бізнес-процесів
Інструменти виявлення	Складно автоматизувати, потрібен manual review governance module

Сценарії експлуатації

Сценарій 1 (flash loan governance). Атакувальник бере flash loan у токенах управління, голосує за пропозицію, яка переказує кошти протоколу йому, повертає кредит. Усе в одній транзакції. Захист це snapshot голосів до flash loan (наприклад, через ERC20Votes з OpenZeppelin).

Сценарій 2 (low quorum exploitation). Якщо для проходження пропозиції потрібно мінімум 4% голосів від загального supply, атакувальник

може просто купити стільки токенів на ринку у непопулярний час (вихідні, нічні години) і провести голосування непомітно.

Сценарій 3 (admin backdoor). Розробники залишили адмін-функцію "для екстреної зупинки", що насправді дозволяє виводити кошти. У разі компрометації ключа адміна або зловмисного наміру усе можна вкрати.

Вимоги для категорії

32.Голосування базувати на snapshot балансів у минулому блоці, виключаючи маніпуляції flash loanом (OpenZeppelin ERC20Votes).

33.Будь-яке рішення governance виконувати через Timelock з затримкою не менше 48 годин, щоб користувачі могли вийти з протоколу при потребі.

34.Адмін-функції віддавати DAO або мультипідпису, не EOA.

35.Впроваджувати роль Guardian, що може скасувати очевидно зловмисну пропозицію, але не може створити власну.

4.9. Категорія 9: Безпека крос-чейн мостів

Мости між блокчейнами стали найкровопротитнішою точкою всієї крипто-екосистеми. З 2022 року атаки на мости призвели до втрат понад 2,5 мільярди доларів (Ronin \$625M, Wormhole \$325M, Nomad \$190M, Poly Network \$611M). Причина в тому, що міст за визначенням мусить мати власну логіку довіри: хтось має авторизовано стверджувати, що в одному блокчейні певна транзакція відбулася. Будь-яка слабкість у цьому механізмі веде до катастрофи.

Імовірність експлуатації	Висока (мости постійно тестуються атакувальниками)
Технічний імпакт	Критичний (повне спустошення мосту)
Бізнес-імпакт	Катастрофічний (часто екзистенційний для проєкту)
Стандарти класифікації	не класифіковано у SWC, нова категорія
Інструменти виявлення	Manual audit обов'язковий, формальна верифікація сигнатур, моніторинг

Сценарії експлуатації

Сценарій 1 (compromised validator keys). Ronin Bridge використовував 9 валідаторів, кворум 5-of-9. Атакувальник через соціальну інженерію (фішинговий PDF з фейкової вакансії) скомпрометував 4 ключі, ще один отримав через інший вектор. 5 з 9 виявилось достатньо. Втрати \$625 мільйонів.

Сценарій 2 (signature verification bug). Wormhole мав логічну помилку у перевірці підписів guardian set. Атакувальник зміг створити підроблений підпис, що пройшов перевірку без участі реальних guardians. Втрати \$325 мільйонів.

Сценарій 3 (misconfigured upgrade). Nomad після оновлення мав значення за замовчуванням, при якому всі повідомлення вважались валідними. Атакувальники просто копіювали успішну транзакцію та заміняли адресу одержувача. Через децентралізовану природу атаки понад 300 окремих адрес встигли вкрати кошти, поки міст не зупинили. Втрати \$190 мільйонів.

Приклад уразливого коду

```
// УРАЗЛИВИЙ варіант: немає chainId у підписаному повідомленні
contract BadBridge {
    address[] public guardians;
    uint256 public threshold;

    function unlock(address to, uint256 amount,
                    bytes[] calldata signatures) external {
        // Хеш не включає chainId, можна replay в іншу мережу
        bytes32 message = keccak256(abi.encode(to, amount));
        uint256 valid = 0;
        for (uint256 i = 0; i < signatures.length; i++) {
            address signer = recover(message, signatures[i]);
            if (isGuardian(signer)) valid++;
        }
        require(valid >= threshold, "Not enough signatures");
        token.transfer(to, amount);
    }
}
```

Вимоги для категорії

- 36.Включати chainId, sourceChainId, contractAddress у кожне підписане повідомлення мосту.
- 37.Замінювати валідаторів через timelock, не моментально. Це дає час виявити компрометацію.
- 38.Впроваджувати rate limiting: ліміт на сумарний вивід за період (наприклад, не більше \$10M за годину).
- 39.Реалізувати circuit breaker, що автоматично призупиняє міст при аномальній активності (виявляється через моніторинг подій).
- 40.Розглянути архітектуру з zk-доказами або optimistic verification, що зменшує довіру до окремих валідаторів.

4.10. Категорія 10: Логування та моніторинг подій

На відміну від традиційних додатків, у блокчейні немає логів сервера. Єдиним способом залишити слід події є emit event у Solidity. Якщо розробник

з лінощів або задля економії газу не випускає event на критичну зміну стану, ця зміна стає невидимою для зовнішніх систем моніторингу. У результаті атака може тривати днями, поки хтось вручну не помітить аномалію.

Імовірність експлуатації	Дуже висока (присутня майже скрізь, окрім топ-протоколів)
Технічний імпакт	Низький-Середній (не дає прямого ламання, але унеможливорює реакцію)
Бізнес-імпакт	Високий (атака триває довше, ніж могла б)
Стандарти класифікації	не класифіковано у SWC
Інструменти виявлення	Slither has detector для missing events, manual review критичних функцій

Сценарії експлуатації

Сценарій 1 (silent drainage). Атакувальник знайшов мікровразливість, що дозволяє виводити невеликі суми. Оскільки функція не випускає event, моніторинг не помічає підозрілої активності. Атака триває тижнями, поки хтось вручну не звірить баланс.

Сценарій 2 (governance attack без сповіщень). Зловмисна пропозиція в DAO створюється у вихідні. Через відсутність моніторингу подій ProposalCreated спільнота не отримує сповіщень у Discord/Telegram і не встигає проголосувати проти.

Сценарій 3 (non-indexed parameters). Розробник додав event, але всі параметри не позначені як indexed. Це робить пошук подій за конкретною адресою чи токеном неможливим без сканування всіх логів, що в продакшені неприйнятно повільно.

Приклад уразливого коду

```
// УРАЗЛИВИЙ варіант: критична зміна без event
contract TreasuryBad {
    address public manager;

    function changeManager(address newManager) external onlyOwner {
        manager = newManager;
        // Немає event! Зовнішні системи не помітять
    }
}
```

Захищений варіант з повноцінними events:

```
contract TreasuryGood {
    address public manager;

    event ManagerChanged(
        address indexed previousManager,
        address indexed newManager,
        address indexed initiator,
```

```

        uint256 timestamp
    );

    function changeManager(address newManager) external onlyOwner {
        require(newManager != address(0), "Zero address");
        address previous = manager;
        manager = newManager;
        emit ManagerChanged(previous, newManager, msg.sender, block.timestamp);
    }
}

```

Вимоги для категорії

41. Випускати event на кожну зміну критичного стану: зміна власника, паузи, оновлення параметрів, переказ великих сум.
42. Включати ключові параметри як indexed (адреси, ID), щоб дозволити швидку фільтрацію через The Graph або подібні індексатори.
43. Підключити моніторинг через Forta Network, OpenZeppelin Defender або Tenderly Alerts з real-time сповіщеннями в Slack/Discord/Telegram.
44. Передбачити окремий dashboard з ключовими метриками протоколу (TVL, кількість транзакцій, аномальні зміни) для оперативної реакції.

5. Зведений набір вимог

Сформовані в попередньому розділі вимоги можна узагальнити у вигляді короткого checklist для розробників та аудиторів.

№	Категорія	Ключова вимога
1	Reentrancy та порушення CEI	Завжди CEI-патерн + ReentrancyGuard на функціях з external calls
2	Контроль доступу до критичних функцій	OpenZeppelin AccessControl, msg.sender замість tx.origin, явний visibility
3	Маніпуляція ціновими оракулами	Тільки Chainlink або TWAP з вікном не менше 30 хвилин, ніколи не спот-ціна DEX
4	MEV та публічний мемпул	Slippage tolerance $\leq 1\%$, інтеграція Flashbots Protect, commit-reveal для критичних дій
5	Криптографічні підписи	EIP-712 з nonce, chainId, deadline; OpenZeppelin ECDSA.sol замість сирого ecrecover
6	DoS та виснаження газу	Pull-pattern замість push, обмеження ітерацій, перевірка gasleft() перед external call
7	Безпека upgradeability	Timelock 48-72 години, multisig на upgrade-функції, перевірка storage layout
8	On-chain governance	Snapshot балансів до flash-loan, timelock на виконання, мінімальний кворум
9	Безпека крос-чейн мостів	chainId у підписаних повідомленнях, rate limiting, circuit breakers на великі переказки
10	Логування та моніторинг	events на всі state changes з indexed-параметрами, real-time

№	Категорія	Ключова вимога
		alerts через Forta/Defender

6. Аналіз гучних атак на dApp

Кожна із сформульованих вимог народилась не з теорії, а з конкретних, дорогих в усіх сенсах помилок. Нижче наведено хронологію найгучніших інцидентів з вказанням категорії та основного уроку.

Атака	Рік	Втрати	Категорія	Урок
The DAO	2016	\$60 млн	Reentrancy	Призвела до хардфорку Ethereum/ETC. Породила CEI-патерн
Parity Multisig	2017	\$300 млн	Access Control	Незахищена бібліотека дозволила анонімному користувачу стати власником і знищити її
bZx	2020	\$8 млн	Oracle (flash loan)	Перша гучна flash loan атака. Спот-ціни DEX не годяться як oracle
Poly Network	2021	\$611 млн	Bridge	Хакер повернув кошти. Показав вразливість крос-чейн мостів
Cream Finance	2021	\$130 млн	Flash loan + Oracle	Маніпуляція цінами на one-block window
Wormhole	2022	\$325 млн	Signature verification	Помилка в перевірці підпису bridge guardians
Ronin Bridge	2022	\$625 млн	Compromised keys	Соціальна інженерія проти валідаторів. 5 з 9 ключів захоплено
Beanstalk	2022	\$182 млн	Governance + flash	Flash loan для проходження зловмисної пропозиції за 13 секунд
Nomad	2022	\$190 млн	Bridge misconfig	Помилка ініціалізації зробила всі повідомлення валідними
Euler Finance	2023	\$197 млн	Logic edge case	Хакер повернув кошти. Edge case у donateToReserves

Аналіз цих кейсів виявляє три закономірності. По-перше, втрати від атак з кожним роком зростають у грошовому вимірі, незважаючи на зростання обізнаності спільноти. По-друге, найдорожчі атаки 2022-2023 років не на самі Ethereum-контракти, а на крос-чейн мости, що підкреслює важливість безпеки мостів як окремої дисципліни. По-третє, flash loans стали універсальним підсилювачем: атакувальник без початкового капіталу може отримати десятки мільйонів на одну транзакцію і використати їх для маніпуляції цінами чи голосування.

7. Висновки

45. Безпека децентралізованих додатків спирається на ту саму базу принципів, що й безпека веб-додатків (контроль доступу, валідація вхідних даних, безпечне зберігання таємниць), але має суттєву специфіку. Незмінність розгорнутого коду означає, що помилка не виправляється патчем. Публічність мемпулу породжує цілий клас MEV-атак. Композиційність DeFi-протоколів через flash loans створює нові комбіновані вектори атак.
46. Із десяти категорій OWASP Top 10:2021 сім мають прямі аналоги серед вразливостей смарт-контрактів, дві (Injection, SSRF) не мають прямих еквівалентів, бо породжені серверною архітектурою, а одна (Vulnerable Components) переноситься з адаптацією на контекст прм-залежностей та форків бібліотек.
47. Сформовано власну ієрархію стандартів dApp-безпеки: DASP Top 10 як аналог OWASP Top 10, SWC Registry як аналог CWE, SCSVS як аналог ASVS. Разом вони покривають як високорівневі категорії, так і конкретні вимоги для верифікації.
48. Запропоновано власний перелік 10 категорій вимог безпеки для Ethereum dApp, який поєднує переваги OWASP-mapping та dApp-специфічних вразливостей. Категорії впорядковано не за алфавітом, а за частотою появи у реальних інцидентах 2020-2024 років. Окремо виділено категорію безпеки крос-чейн мостів, що відсутня у класичних dApp-стандартах попередніх років, але стала найбільш руйнівною за грошовими втратами.
49. Запропоновані вимоги мають конкретний характер і прив'язані до існуючих інструментів: OpenZeppelin Contracts для базової безпеки, Slither та Mythril для статичного аналізу, Echidna для invariant-тестування, Foundry для fuzzing, Forta та OpenZeppelin Defender для real-time моніторингу, Gnosis Safe для мультипідпису, Chainlink VRF для випадковості, EIP-712 для підписів.
50. Історія атак на Ethereum-екосистему (з накопиченими втратами понад 5 мільярдів доларів станом на 2024 рік) показує, що формальні вимоги безпеки не є достатніми без культури безпеки в команді: незалежні аудити перед розгортанням, bug bounty з винагородами пропорційними TVL, постійний моніторинг і готовність до інцидентів. Ці елементи відповідають аналогічним практикам у Web2 (SDLC, OWASP SAMM), але мають свою специфіку для блокчейн-середовища.

Загальний підсумок: безпека dApp це не просто переклад OWASP на мову Solidity. Це окрема дисципліна з власними категоріями вразливостей, власними інструментами, власними економічними загрозами (flash loans, MEV) і своєю історією дорогих помилок. Запропоновані 10 категорій вимог можна використовувати як стартовий чек-лист, але реальний рівень безпеки досягається тільки комбінацією дотримання цих вимог, незалежного аудиту та постійного моніторингу.

8. Список використаних джерел

1. OWASP Top 10:2021. Open Web Application Security Project. URL: <https://owasp.org/Top10/>
2. OWASP Application Security Verification Standard (ASVS) v4.0.3. URL: <https://owasp.org/www-project-application-security-verification-standard/>
3. DASP Top 10 (Decentralized Application Security Project). NCC Group, 2018. URL: <https://dasp.co/>
4. SWC Registry: Smart Contract Weakness Classification. URL: <https://swcregistry.io/>
5. Smart Contract Security Verification Standard (SCSVS). URL: <https://securing.github.io/SCSVS/>
6. ConsenSys Diligence. Ethereum Smart Contract Best Practices. URL: <https://consensys.github.io/smart-contract-best-practices/>
7. Trail of Bits. Building Secure Contracts. URL: <https://secure-contracts.com/>
8. OpenZeppelin Contracts Documentation. URL: <https://docs.openzeppelin.com/contracts/>
9. Atzei N., Bartoletti M., Cimoli T. A Survey of Attacks on Ethereum Smart Contracts. POST 2017, LNCS vol. 10204.
10. Daian P., Goldfeder S., Kell T. Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges. IEEE S&P, 2020.
11. Qin K., Zhou L., Livshits B., Gervais A. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. Financial Cryptography 2021.
12. Zhou L. et al. SoK: Decentralized Finance (DeFi) Attacks. IEEE S&P, 2023.
13. Rekt News. Хроніка атак на DeFi. URL: <https://rekt.news/>
14. Chainalysis. The Chainalysis 2024 Crypto Crime Report. URL: <https://www.chainalysis.com/>